บทที่ 7: ความรู้เบื้องต้นเกี่ยวกับ State: StatefulWidget & setState

7.1 บทนำ: การทำให้แอปโต้ตอบได้

จนถึงตอนนี้ ส่วนติดต่อผู้ใช้ที่เราสร้างขึ้นส่วนใหญ่เป็นแบบคงที่ (static) เราได้จัดเรียงข้อความ, รูปภาพ, ไอคอน และปุ่ม แต่มันยังไม่ได้ *ตอบสนอง* แบบไดนามิกต่อการป้อนข้อมูลของผู้ใช้หรือข้อมูลที่เปลี่ยนแปลงภายในแอปเอง (ยกเว้นการพิมพ์ข้อความใน onPressed) อย่างไรก็ตาม แอปพลิเคชันในโลกแห่งความเป็นจริงนั้นมีการเปลี่ยนแปลงอยู่เสมอ ตัวนับเพิ่มค่า, รายการอัปเดต, ฟอร์มแสดงข้อผิดพลาดในการตรวจสอบความถูกต้อง, โปรไฟล์ผู้ใช้โหลดข้อมูล – สิ่งต่างๆ เปลี่ยนแปลงไปตามการโต้ตอบของผู้ใช้หรือเหตุการณ์ภายนอก

พฤติกรรมไดนามิกนี้ถูกจัดการผ่าน State พูดง่ายๆ State คือ ข้อมูลใดๆ ที่มีอยู่ตลอดเวลาและสามารถเปลี่ยนแปลงได้ ซึ่งอาจส่งผลกระทบต่อ UI ตัวอย่างเช่น ค่าปัจจุบันของตัวนับ, ข้อความในช่องป้อนข้อมูล, การติ๊กถูกของ checkbox, หรือรายการผลลัพธ์ที่ดึงมาจากเซิร์ฟเวอร์

ใน Flutter วิดเจ็ตเองนั้นไม่เปลี่ยนรูป (immutable - การกำหนดค่าไม่สามารถเปลี่ยนแปลงได้หลังจากการสร้าง) แล้ว UI จะอัปเดตได้อย่างไรเมื่อข้อมูลเปลี่ยนแปลง? Flutter แนะนำแนวคิดของ StatefulWidget และออบเจกต์ State ที่เกี่ยวข้อง

บทนี้มีความสำคัญอย่างยิ่ง เนื่องจากเป็นการแนะนำกลไกพื้นฐานสำหรับการจัดการการเปลี่ยนแปลงและการโต้ตอบใน Flutter เราจะ:

- กำหนดความหมายของ "State" ในบริบทของ Flutter
- แยกความแตกต่างระหว่าง StatelessWidget (ที่เราใช้มาจนถึงตอนนี้) และ StatefulWidget
- ทำความเข้าใจความสัมพันธ์ระหว่าง StatefulWidget และออบเจกต์ State ของมัน
- เรียนรู้เกี่ยวกับวงจรชีวิต (lifecycle) ของออบเจกต์ State (เมธอดสำคัญ เช่น initState, build, dispose)
- ฝึกฝนการใช้เมธอด setState() เพื่อแจ้ง Flutter ว่า state มีการเปลี่ยนแปลงและ UI จำเป็นต้องถูกสร้างใหม่ (rebuild)
- แยกแยะระหว่าง ephemeral state (สถานะเฉพาะที่ของวิดเจ็ตเดียว) และ app state (สถานะที่แชร์ข้ามวิดเจ็ต)
- กล่าวถึงการยก state ขึ้น (lifting state up) โดยใช้ callbacks สั้นๆ

เมื่อจบบทนี้ คุณจะเข้าใจวิธีสร้างวิดเจ็ตที่สามารถเปลี่ยนรูปลักษณ์และพฤติกรรมเมื่อเวลาผ่านไปเพื่อตอบสนองต่อการกระทำของผู้ใช้หรือเหตุการณ์อื่นๆ ซึ่งเป็นแกนหลักของแอปพลิเคชัน Flutter ที่โต้ตอบได้

7.2 State คืออะไร?

ใน Flutter, state คือ ข้อมูลที่สามารถเปลี่ยนแปลงได้ในระหว่างช่วงชีวิตของวิดเจ็ต และอาจส่งผลกระทบต่อส่วนติดต่อผู้ใช้ [1]

ลองนึกถึงองค์ประกอบ UI ทั่วไป:

- Checkbox: state ของมันคือสถานะว่าถูกเลือกอยู่หรือไม่ (bool)
- ช่องป้อนข้อความ: state ของมันรวมถึงข้อความปัจจุบันที่ผู้ใช้ป้อน (String)
- Slider: state ของมันคือค่าตัวเลขปัจจุบัน (double)
- ปุ่มนับ: state ของมันคือจำนวนนับปัจจุบัน (int)
- ลิสต์แสดงผลการค้นหา: state ของมันคือลิสต์ของผลลัพธ์ (List<ResultObject>) ซึ่งเปลี่ยนแปลงตามคำค้นหา

เมื่อใดก็ตามที่ข้อมูลนี้เปลี่ยนแปลง โดยหลักการแล้ว UI ควรจะอัปเดตเพื่อสะท้อน state ใหม่

7.3 StatelessWidget เปรียบเทียบกับ StatefulWidget

Flutter มีวิดเจ็ตหลักสองประเภทเพื่อจัดการกับการมีอยู่หรือไม่มีอยู่ของ state ภายใน [2]:

7.3.1 StatelessWidget

- คำจำกัดความ: วิดเจ็ตที่การกำหนดค่าไม่เปลี่ยนรูป (immutable) และถูกกำหนดโดยสมบูรณ์จาก arguments ที่ส่งเข้ามาใน constructor และ BuildContext ที่มันถูกสร้างขึ้น มัน ไม่มี state ภายในใดๆ ที่เปลี่ยนแปลงตลอดช่วงชีวิตของมัน [3]
- ลักษณะ:
  - เมื่อสร้างแล้ว จะไม่เปลี่ยนแปลงตามการโต้ตอบของผู้ใช้หรือการอัปเดตข้อมูลภายใน *ตัวมันเอง*
  - เมธอด build() ของมันถูกเรียกเฉพาะเมื่อวิดเจ็ตแม่ของมันสร้างใหม่ หรือเมื่อ constructor arguments เปลี่ยนแปลง (หากมันขึ้นอยู่กับ InheritedWidget)
  - ตัวอย่าง: Text, Icon, Container (เมื่อคุณสมบัติคงที่), Padding, Center, Row, Column รวมถึงวิดเจ็ตที่คุณสร้างขึ้นเองซึ่งแสดงข้อมูลที่ส่งเข้ามาเท่านั้น
- กรณีใช้งาน: สำหรับองค์ประกอบ UI ที่ไม่จำเป็นต้องเปลี่ยนแปลงแบบไดนามิกตามปัจจัยภายในหลังจากที่ถูกสร้างขึ้นครั้งแรก เหมาะสำหรับการแสดงข้อมูลคงที่หรือการจัดโครงสร้างเลย์เอาต์

7.3.2 StatefulWidget

- คำจำกัดความ: วิดเจ็ตที่ สามารถรักษา state ที่อาจเปลี่ยนแปลงได้ในระหว่างช่วงชีวิตของมัน ทำให้ UI ของมันถูกสร้างใหม่ได้ [4]
- ลักษณะ:
  - ไม่มีเมธอด build() โดยตรง แต่จะสร้างออบเจกต์ State แยกต่างหาก
  - ออบเจกต์ State เก็บข้อมูล state ที่เปลี่ยนแปลงได้ (mutable) และมีเมธอด build()
  - เมื่อ state ภายในเปลี่ยนแปลง (เกิดจากการโต้ตอบของผู้ใช้, ตัวจับเวลา, การตอบสนองจากเครือข่าย ฯลฯ) คุณจะเรียกเมธอด setState() ภายในออบเจกต์ State
  - การเรียก setState() จะบอก Flutter framework ว่า state มีการเปลี่ยนแปลง และ framework จะกำหนดเวลาสร้าง subtree ของวิดเจ็ตนั้นใหม่โดยการเรียกเมธอด build() ของออบเจกต์ State อีกครั้ง
  - ตัวอย่าง: Checkbox, TextField, Slider รวมถึงวิดเจ็ตที่คุณสร้างขึ้นเองที่ต้องการจัดการข้อมูลภายในที่มีผลต่อรูปลักษณ์หรือพฤติกรรมเมื่อเวลาผ่านไป (เช่น ตัวอย่างตัวนับของเรา)
- กรณีใช้งาน: สำหรับองค์ประกอบ UI ใดๆ ที่ต้องเปลี่ยนแปลงแบบไดนามิกตามการโต้ตอบของผู้ใช้หรือการอัปเดตข้อมูลภายใน

ความแตกต่างที่สำคัญ: StatelessWidget อธิบายส่วนของ UI ตามการกำหนดค่าที่ไม่เปลี่ยนรูป StatefulWidget อธิบายส่วนของ UI ที่สามารถเปลี่ยนแปลงได้ตาม state ที่เปลี่ยนแปลงได้ซึ่งเก็บไว้ภายในออบเจกต์ State ที่เกี่ยวข้อง

7.4 คู่หู StatefulWidget และ State

StatefulWidget เองนั้นยังคงไม่เปลี่ยนรูป (immutable) (เหมือน StatelessWidget) state ที่ *เปลี่ยนแปลงได้* (mutable) จะถูกเก็บไว้ในออบเจกต์แยกต่างหาก ซึ่งตามธรรมเนียมจะตั้งชื่อว่า _YourWidgetNameState ซึ่งสืบทอด (extends) มาจากคลาส State<YourWidgetName> [4]

นี่คือโครงสร้างพื้นฐาน:

```dart
import 'package:flutter/material.dart';

// 1. คลาส StatefulWidget (immutable)
class MyCounterWidget extends StatefulWidget {
  final String title; // ตัวอย่าง configuration ที่ส่งเข้ามา

  // Constructor เป็น const เพราะ configuration ของวิดเจ็ตเองไม่เปลี่ยนรูป
```

```dart
  const MyCounterWidget({super.key, required this.title});

  // เมธอดสำคัญ: สร้างออบเจกต์ State ที่เปลี่ยนแปลงได้
  @override
  State<MyCounterWidget> createState() => _MyCounterWidgetState();
}

// 2. คลาส State (mutable, เกี่ยวข้องกับ MyCounterWidget)
// เครื่องหมายขีดล่าง (_) ทำให้คลาสเป็น private สำหรับไลบรารีนี้ (ไฟล์นี้)
class _MyCounterWidgetState extends State<MyCounterWidget> {
  // --- ตัวแปร State ---
  int _counter = 0; // ตัวแปร state ที่เปลี่ยนแปลงได้

  // --- เมธอดช่วยเหลือ (ที่อาจเปลี่ยน state) ---
  void _incrementCounter() {
    // setState() สำคัญมากที่นี่!
    setState(() {
      // การเรียกนี้บอก Flutter ว่า state มีการเปลี่ยนแปลง
      // และเมธอด build() ต้องทำงานอีกครั้ง
      // เราแก้ไข state *ภายใน* callback ของ setState
      _counter++;
      print('ตัวนับเพิ่มเป็น: $_counter'); // สำหรับดีบัก
    });
  }

  // --- เมธอด Lifecycle (ทางเลือก แต่ใช้บ่อย) ---
  @override
  void initState() {
    super.initState();
    // ถูกเรียกครั้งเดียวเมื่อออบเจกต์ state ถูกสร้างขึ้นครั้งแรก
    // เป็นที่ที่ดีสำหรับการตั้งค่าครั้งเดียว เช่น การกำหนดค่าเริ่มต้นให้ controllers
    // หรือการสมัครรับข้อมูล streams
    print('Counter State เริ่มต้นแล้ว!');
    // กำหนดค่า state เริ่มต้นตาม properties ของ widget หากจำเป็น:
    // _counter = widget.initialValue; // สมมติว่า MyCounterWidget มี initialValue
  }

  @override
  void dispose() {
    // ถูกเรียกเมื่อออบเจกต์ state ถูกลบออกจาก tree อย่างถาวร
    // สำคัญสำหรับการล้างข้อมูล: ยกเลิก timers, ยกเลิกการสมัคร streams,
    // dispose controllers (เช่น TextEditingController)
    print('Counter State ถูก Dispose แล้ว!');
    super.dispose(); // เรียก super.dispose() ตอนท้ายเสมอ
  }

  // --- เมธอด Build (จำเป็น) ---
```

```
  @override
  Widget build(BuildContext context) {
    // เมธอดนี้ทำงานครั้งแรก และ *ทุกครั้ง* ที่ setState ถูกเรียก
    // ควรสร้าง UI ตาม state *ปัจจุบัน*
    print('กำลังสร้าง UI ของ Counter Widget... ค่าปัจจุบัน: $_counter');

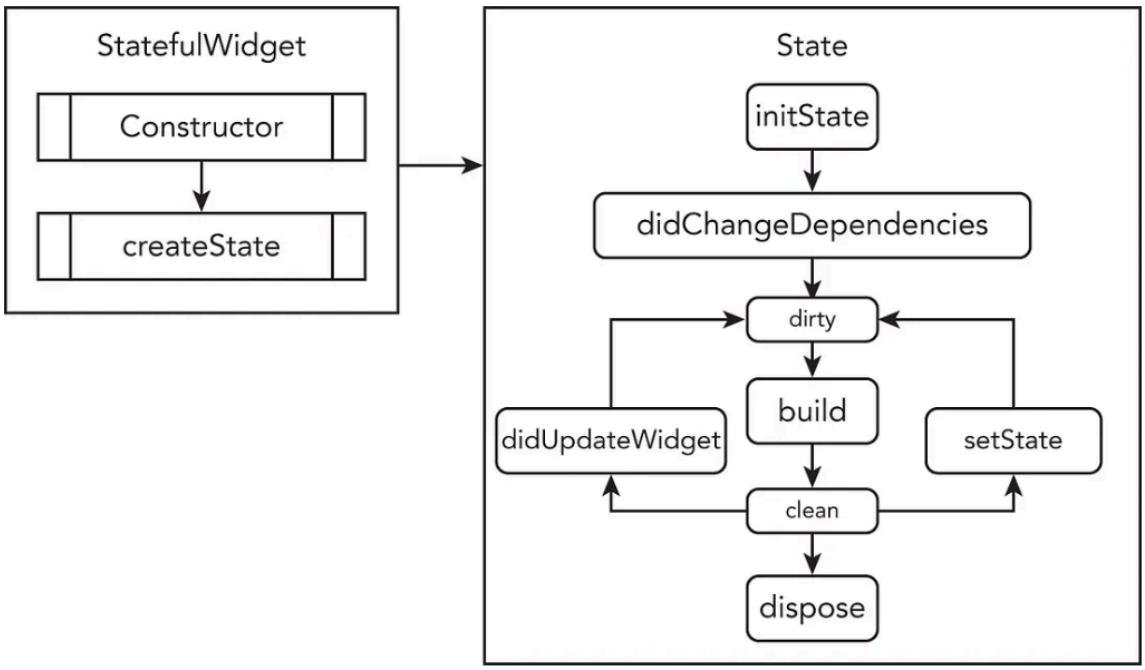
    // เข้าถึง properties ของ widget โดยใช้ 'widget.'
    String currentTitle = widget.title;

    return Scaffold( // ตัวอย่างโครงสร้าง UI
      appBar: AppBar(title: Text(currentTitle)),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text('คุณกดปุ่มนี้ไปแล้ว:'),
            Text(
              '$_counter', // แสดงตัวแปร state ปัจจุบัน
              style: Theme.of(context).textTheme.headlineMedium,
            ),
          ],
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _incrementCounter, // เรียกเมธอดของเราเมื่อกด
        tooltip: 'Increment',
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

ประเด็นสำคัญ:

- StatefulWidget (MyCounterWidget) นั้นเรียบง่ายและส่วนใหญ่รับผิดชอบในการสร้างออบเจกต์ State ของมันผ่าน createState()
- ออบเจกต์ State (_MyCounterWidgetState) เก็บข้อมูลที่เปลี่ยนแปลงได้จริง (_counter)
- ออบเจกต์ State มีเมธอด build() ซึ่งใช้ state ปัจจุบัน (_counter) เพื่อสร้าง UI
- เมื่อเกิดเหตุการณ์ขึ้น (เช่น การกดปุ่มเรียก _incrementCounter) เมธอดจะแก้ไขตัวแปร state (_counter++) *ภายใน* การเรียก setState()
- setState() ทำเครื่องหมายวิดเจ็ตว่า "dirty" (มีการเปลี่ยนแปลง) และกำหนดเวลาให้มีการ rebuild Flutter จะเรียกเมธอด build() อีกครั้ง และ UI จะอัปเดตเพื่อสะท้อนค่าใหม่ของ _counter

7.5 วงจรชีวิตของออบเจกต์ State (State Object Lifecycle)



ออบเจกต์ State ที่เกี่ยวข้องกับ StatefulWidget มีวงจรชีวิต หมายความว่ามันผ่านช่วงต่างๆ ตั้งแต่การสร้างไปจนถึงการทำลาย การทำความเข้าใจสิ่งเหล่านี้ช่วยให้จัดการทรัพยากรได้อย่างถูกต้อง [5] เมธอด lifecycle ที่สำคัญ ได้แก่:

1. createState(): ถูกเรียกโดย StatefulWidget เพื่อสร้างออบเจกต์ State (คุณ implement เมธอดนี้ในคลาส StatefulWidget)
2. initState(): ถูกเรียก *เพียงครั้งเดียว* เมื่อออบเจกต์ State ถูกแทรกเข้าไปใน widget tree เป็นครั้งแรก นี่คือที่สำหรับการกำหนดค่าเริ่มต้นครั้งเดียว:
   - การสมัครรับข้อมูล streams หรือ change notifiers
   - การกำหนดค่าเริ่มต้นให้ controllers (เช่น TextEditingController, AnimationController)
   - การตั้งค่าที่ขึ้นอยู่กับ context (แต่บางครั้ง didChangeDependencies เหมาะสมกว่า)
   - สำคัญ: เรียก super.initState(); ที่จุดเริ่มต้นของการ implement initState ของคุณเสมอ
3. didChangeDependencies(): ถูกเรียกทันทีหลัง initState ในครั้งแรก และหลังจากนั้นเมื่อใดก็ตามที่ InheritedWidget ที่ State object นี้ขึ้นอยู่ มีการเปลี่ยนแปลง มีประโยชน์สำหรับการตั้งค่าที่ต้องพึ่งพาข้อมูล BuildContext หรือ InheritedWidget
4. build(BuildContext context): ถูกเรียกหลัง initState (และ didChangeDependencies) และ ทุกครั้งที่ setState() ถูกเรียก หรือเมื่อ framework ตัดสินใจว่าวิดเจ็ตต้องการการ rebuild เมธอดนี้ *ต้อง* ส่งคืนวิดเจ็ต และโดยหลักการแล้วควรจะรวดเร็วและไม่มี side effects เพียงแค่สร้าง UI ตาม state และ configuration ปัจจุบันของวิดเจ็ต
5. didUpdateWidget(covariant OldWidget oldWidget): ถูกเรียกเมื่อวิดเจ็ตแม่ rebuild และให้ *อินสแตนซ์ใหม่* ของ StatefulWidget ที่เกี่ยวข้อง (อาจมี configuration ที่แตกต่างกัน) แต่มี runtimeType *เดิม* framework จะใช้ออบเจกต์ State ที่มีอยู่ซ้ำ คุณสามารถเปรียบเทียบ widget (configuration ใหม่) กับ oldWidget เพื่อตอบสนองต่อการเปลี่ยนแปลงได้ (เช่น รีเซ็ต state, อัปเดต subscriptions)
6. setState(VoidCallback fn): เมธอดสำคัญที่คุณเรียกเพื่อส่งสัญญาณว่า state ภายในมีการเปลี่ยนแปลง ฟังก์ชัน callback fn จะถูกดำเนินการทันที จากนั้น framework จะกำหนดเวลาเรียก build() คุณ *ต้อง* แก้ไขตัวแปร state *ภายใน* callback ของ setState (หรือก่อนเรียกใช้ หากการแก้ไขนั้นไม่ได้ขึ้นอยู่กับการเปลี่ยนแปลง state อื่นๆ ภายใน callback) [6]
7. deactivate(): ถูกเรียกเมื่อออบเจกต์ State ถูกลบออกจาก tree แต่มันอาจถูกแทรกกลับเข้าไปที่อื่นได้ (เช่น เมื่อย้ายวิดเจ็ตโดยใช้ GlobalKey) Framework เรียกเมธอดนี้น้อยกว่า dispose
8. dispose(): ถูกเรียกเมื่อออบเจกต์ State ถูกลบออกจาก tree *อย่างถาวร* นี่คือที่สำหรับการล้างข้อมูลครั้งสุดท้าย:
   - ยกเลิกการสมัคร streams, timers, notifiers

- Dispose controllers (_myController.dispose();) การไม่ dispose controllers อาจทำให้เกิด memory leaks
- สำคัญ: เรียก super.dispose(); ที่ *ตอนท้าย* ของการ implement dispose ของคุณเสมอ

สำหรับการจัดการ state พื้นฐาน คุณจะใช้ initState, build, setState, และ dispose เป็นหลัก

7.6 setState(): ตัวกระตุ้นการอัปเดต UI

ขอย้ำถึงความสำคัญของ setState() อีกครั้ง การเปลี่ยนค่าของตัวแปร state (เช่น _counter++) *นอก* การเรียก setState() จะไม่ ทำให้ UI อัปเดตตามที่เห็น

```
// วิธีที่ผิด - UI จะไม่อัปเดต
void _incrementCounterWrong() {
  _counter++; // ตัวแปร State เปลี่ยน...
  print('ตัวนับเปลี่ยน (ผิด): $_counter');
  // ... แต่ Flutter ไม่รู้ว่าต้อง rebuild!
}

// วิธีที่ถูก - UI จะอัปเดต
void _incrementCounterCorrect() {
  setState(() { // บอก Flutter ว่ามีบางอย่างเปลี่ยน
    _counter++; // แก้ไข state ภายใน callback
    print('ตัวนับเปลี่ยน (ถูก): $_counter');
  });
  // Flutter กำหนดเวลาเรียก build() หลังจาก callback นี้เสร็จสิ้น
}
```

ทำไมต้องเรียก setState? มันทำเครื่องหมาย element ของวิดเจ็ตว่า dirty เพิ่มเข้าไปในลิสต์ของ elements ที่ต้อง rebuild ในระหว่างรอบการเรนเดอร์เฟรมถัดไป สิ่งนี้ทำให้มั่นใจได้ว่าการอัปเดต UI จะถูกรวม (batch) อย่างมีประสิทธิภาพ แทนที่จะเกิดขึ้นแบบสุ่ม [6]

7.7 Ephemeral State กับ App State

เมื่อแอปพลิเคชันเติบโตขึ้น การจำแนกประเภทของ state จะมีประโยชน์ [7]:

1. Ephemeral State (Local State, UI State):
   - State ที่อยู่ภายในวิดเจ็ตเดียวอย่างเรียบร้อย (หรืออาจส่งผ่านระหว่าง parent และ child โดยตรง)
   - ไม่จำเป็นต้องเข้าถึงโดยส่วนอื่นๆ ที่อยู่ห่างไกลใน widget tree
   - ตัวอย่าง: หน้าปัจจุบันใน PageView, ความคืบหน้าของแอนิเมชัน, แท็บที่เลือกใน BottomNavigationBar, ข้อความใน TextField ก่อนส่ง
   - การจัดการ: มักจะจัดการได้อย่างสมบูรณ์แบบโดยใช้ StatefulWidget และ setState()
2. App State (Shared State):
   - State ที่ต้องแชร์ข้ามส่วนต่างๆ ของแอปพลิเคชัน อาจจะข้ามหลายหน้าจอหรือวิดเจ็ตที่อยู่ห่างกันมาก
   - ต้องเก็บรักษาไว้ระหว่าง session (บางครั้ง)
   - ตัวอย่าง: ข้อมูลการล็อกอินของผู้ใช้, เนื้อหารถเข็นสินค้า, การตั้งค่าหรือ theme ทั่วทั้งแอป, การแจ้งเตือน
   - การจัดการ: แม้ว่าในทางเทคนิคจะทำได้ด้วย setState และ callbacks ที่ส่งลงไปตาม tree ("lifting state up") แต่วิธีนี้จะยุ่งยากและเกิดข้อผิดพลาดได้ง่ายในแอปขนาดใหญ่ นี่คือจุดที่ แพ็กเกจการจัดการ State โดยเฉพาะ (เช่น Provider, Riverpod, BLoC, GetX ฯลฯ) กลายเป็นสิ่งจำเป็น

สำหรับตอนนี้ เราจะเน้นไปที่การจัดการ ephemeral state โดยใช้ StatefulWidget และ setState() เราจะจัดการ App State ด้วยแพ็กเกจอย่าง Provider ในบทต่อๆ ไป

7.8 การยก State ขึ้น (Lifting State Up - บทนำสั้นๆ)

จะทำอย่างไรถ้า child widget ต้องการเปลี่ยน state ที่ถูกเก็บไว้โดย parent (หรือ grandparent) ของมัน? child ไม่สามารถเรียก setState() ของ parent โดยตรงได้ รูปแบบทั่วไปคือ "การยก state ขึ้น" (lifting state up) [8]:

1. State ถูกเก็บไว้ใน ancestor widget ร่วม (ซึ่งต้องเป็น StatefulWidget)
2. Ancestor ส่ง *ฟังก์ชัน callback* ลงไปยัง child widget ผ่าน constructor ของมัน
3. เมื่อเกิดเหตุการณ์ใน child widget ที่ต้องการเปลี่ยน state ที่แชร์กัน child จะเรียกฟังก์ชัน callback ที่ได้รับมา
4. ฟังก์ชัน callback (ที่กำหนดใน ancestor) เรียก setState() ภายในออบเจกต์ State ของ ancestor เพื่ออัปเดต state
5. Ancestor rebuild ส่งข้อมูล state ใหม่ลงไปยัง children ที่เกี่ยวข้อง (รวมถึงอันที่เริ่มต้นการเปลี่ยนแปลง) ทำให้พวกมัน rebuild ด้วยข้อมูลที่อัปเดตแล้ว

```dart
// Parent Widget (Stateful)
class ParentWidget extends StatefulWidget {
  const ParentWidget({super.key});
  @override
  State<ParentWidget> createState() => _ParentWidgetState();
}

class _ParentWidgetState extends State<ParentWidget> {
  bool _isActive = false; // State เก็บโดย parent

  void _handleActiveChange(bool newValue) {
    setState(() { // Parent อัปเดต state ผ่าน callback
      _isActive = newValue;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('สถานะ Parent คือ: ${_isActive ? "Active" : "Inactive"}'),
        ChildWidget( // ส่ง state ลงไป
          isActive: _isActive,
          onActiveChanged: _handleActiveChange, // ส่ง callback ลงไป
        ),
      ],
    );
  }
}

// Child Widget (อาจเป็น Stateless หรือ Stateful)
class ChildWidget extends StatelessWidget {
```

```dart
  final bool isActive;
  final ValueChanged<bool> onActiveChanged; // ฟังก์ชันสำหรับเรียก parent

  const ChildWidget({
    super.key,
    required this.isActive,
    required this.onActiveChanged,
  });

  @override
  Widget build(BuildContext context) {
    return Switch( // วิดเจ็ตตัวอย่างที่เปลี่ยนค่า boolean
      value: isActive,
      onChanged: onActiveChanged, // เรียก callback ของ parent เมื่อมีการเปลี่ยนแปลง
    );
  }
}
```

รูปแบบนี้เป็นพื้นฐานแต่สามารถซับซ้อนได้กับ widget tree ที่ลึก ซึ่งเป็นเหตุผลว่าทำไมแพ็กเกจการจัดการ state จึงมักเป็นที่ต้องการสำหรับ app state

7.9 สรุปบทเรียนและสิ่งที่จะเรียนรู้ต่อไป

บทนี้ได้แนะนำแนวคิดหลักสำหรับการจัดการการเปลี่ยนแปลงและการโต้ตอบใน Flutter:

- State: ข้อมูลที่สามารถเปลี่ยนแปลงได้เมื่อเวลาผ่านไปและส่งผลต่อ UI
- StatelessWidget: สำหรับองค์ประกอบ UI ที่มี configuration ไม่เปลี่ยนรูป
- StatefulWidget: สำหรับองค์ประกอบ UI ที่รูปลักษณ์หรือพฤติกรรมขึ้นอยู่กับ state ภายในที่เปลี่ยนแปลงได้ ทำงานร่วมกับออบเจกต์ State แยกต่างหาก
- ออบเจกต์ State: เก็บตัวแปร state ที่เปลี่ยนแปลงได้และเมธอด build() มี lifecycle (initState, dispose ฯลฯ)
- setState(): เมธอดสำคัญที่เรียกภายในออบเจกต์ State เพื่อแจ้ง Flutter ว่า state มีการเปลี่ยนแปลง กระตุ้นให้ UI rebuild
- Ephemeral vs. App State: การแยกความแตกต่างระหว่าง state เฉพาะที่ (จัดการโดย setState) กับ state ที่แชร์ (มักต้องการแพ็กเกจการจัดการ state)
- Lifting State Up: รูปแบบที่ใช้ callbacks เพื่อให้ child widgets สามารถแก้ไข state ที่เก็บไว้ใน ancestor widgets

ตอนนี้คุณเข้าใจความแตกต่างพื้นฐานระหว่าง stateless และ stateful widgets และวิธีใช้ setState เพื่อทำให้ UI ของคุณตอบสนองต่อการเปลี่ยนแปลงแล้ว

กิจกรรมปฏิบัติการสำหรับบทที่ 7:

แล็บจะเน้นไปที่การสร้างและจัดการ state ด้วย StatefulWidget:

1. แปลง Stateless เป็น Stateful: นำ StatelessWidget แบบง่ายๆ (เช่น อันที่แสดงข้อความ) มา refactor เป็น StatefulWidget และคลาส State ที่เกี่ยวข้อง
2. สร้างตัวนับ: สร้างแอปพลิเคชันตัวนับคลาสสิกขึ้นมาใหม่โดยใช้ StatefulWidget, setState, และ ElevatedButton
3. วิดเจ็ตโต้ตอบได้: สร้างวิดเจ็ต (เช่น Container) ที่เปลี่ยนสีหรือเนื้อหาข้อความเมื่อถูกแตะ โดยใช้ setState
4. การใช้ initState: แก้ไขแบบฝึกหัดก่อนหน้าอันใดอันหนึ่งเพื่อกำหนดค่าเริ่มต้นให้ตัวแปร state ภายในเมธอด initState
5. การใช้ dispose (ฝึกแนวคิด): เพิ่ม TextEditingController ให้กับ StatefulWidget กำหนดค่าเริ่มต้นใน initState และ *อย่าลืม* เรียก

_controller.dispose() ในเมธอด dispose (แม้ว่าจะยังไม่ได้ใช้ controller อย่างเต็มที่ เพื่อฝึกรูปแบบ)

6. (ทางเลือก) Lifting State Up: สร้างตัวอย่าง Parent/Child callback ที่แสดงในหัวข้อ 7.8

บทต่อไป:

เมื่อเราสามารถจัดการการเปลี่ยนแปลง state พื้นฐานได้แล้ว เราต้องการวิธีให้ผู้ใช้ *ป้อน* ข้อมูลได้อย่างมีประสิทธิภาพมากกว่าแค่การแตะปุ่ม บทที่ 8: การจัดการ Input ของผู้ใช้และฟอร์ม จะครอบคลุม TextField, TextFormField, การตรวจสอบความถูกต้อง (validation), และวิธีรวบรวม input ที่มีโครงสร้างโดยใช้วิดเจ็ต Form

เอกสารอ้างอิง (References)

[1] Flutter Team, "Add interactivity to your Flutter app," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/ui/interactivity

[2] Flutter Team, "StatefulWidget class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/StatefulWidget-class.html

[3] Flutter Team, "StatelessWidget class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/StatelessWidget-class.html

[4] Flutter Team, "State class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/State-class.html

[5] Flutter Team, "State class - Lifecycle," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/State-class.html#lifecycle

[6] Flutter Team, "State.setState method," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/State/setState.html

[7] Flutter Team, "State management introduction," *Flutter Documentation*. Accessed: Apr. 29, 2025. [Online]. Available: https://docs.flutter.dev/data-and-backend/state-mgmt/options#ephemeral-state-vs-app-state

[8] Flutter Team, "Lifting state up," *Flutter Documentation*. Accessed: Apr. 29, 2025. [Online]. Available: https://docs.flutter.dev/data-and-backend/state-mgmt/options#lifting-state-up

ปฏิบัติการบทที่ 7: การจัดการ State ด้วย StatefulWidget

วัตถุประสงค์: เพื่อทำความเข้าใจและฝึกฝนการสร้าง StatefulWidget, การจัดการ state โดยใช้ setState, และทำความเข้าใจวงจรชีวิตพื้นฐานของ state

เวลาโดยประมาณ: 2 ชั่วโมง

สิ่งที่ต้องมี: เสร็จสิ้นปฏิบัติการบทที่ 6 (คุ้นเคยกับวิดเจ็ตพื้นฐานและเลย์เอาต์), ติดตั้ง VS Code (หรือ Android Studio), Emulator/อุปกรณ์จริงกำลังทำงาน

การตั้งค่า: เตรียมหน้าจอสำหรับแล็บ

1. คำแนะนำ: เปิดโปรเจกต์ Flutter ของคุณ ตั้งชื่อว่า ‘stateful_app’ ใน VS Code ไปที่ lib/main.dart
2. คำแนะนำ: แก้ไขวิดเจ็ต MyHomePage คง Scaffold และ AppBar ไว้ แต่ทำให้ body ในตอนแรกมีวิดเจ็ต Center พร้อม Text placeholder ง่ายๆ

```
// ภายในเมธอด build ของ MyHomePage (ตรวจสอบให้แน่ใจว่าตอนนี้ MyHomePage เป็น StatelessWidget ไปก่อน)
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('ปฏิบัติการบทที่ 7 - State'), // อัปเดต title
      backgroundColor: Colors.purple, // เปลี่ยนสี
    ),
    body: const Center(
       child: Text('เริ่มทดลอง State ที่นี่!'), // Placeholder
    ),
  );
}
```

3. คำแนะนำ: รันแอป (Ctrl+F5 หรือ Cmd+F5)
    - ผลลัพธ์ที่คาดหวัง (UI): คุณเห็นแถบ AppBar พร้อม title และข้อความ placeholder อยู่กึ่งกลางใน body

แบบฝึกหัดที่ 1: แปลง Stateless เป็น Stateful

1. คำแนะนำ: ทำให้ MyHomePage เป็น stateful เปลี่ยน MyHomePage ให้ extends StatefulWidget แทน StatelessWidget
2. คำแนะนำ: ย้ายเมธอด build ออก ลบเมธอด build ออกจาก MyHomePage
3. คำแนะนำ: สร้างเมธอด createState() ใน MyHomePage เมธอดนี้ควรส่งคืนอินสแตนซ์ของคลาส State ใหม่ (เช่น _MyHomePageState)

```
class MyHomePage extends StatefulWidget { // เปลี่ยนเป็น StatefulWidget
  const MyHomePage({super.key});

  @override
  State<MyHomePage> createState() => _MyHomePageState(); // สร้าง createState
}
```

4. คำแนะนำ: สร้างคลาส _MyHomePageState ใต้ MyHomePage กำหนด class _MyHomePageState extends State<MyHomePage>
5. คำแนะนำ: ย้ายเมธอด build (รวมถึง Scaffold และเนื้อหาภายใน) จาก StatelessWidget เดิมมาไว้ในคลาส _MyHomePageState ใหม่นี้

```
class _MyHomePageState extends State<MyHomePage> { // คลาส State
  // ตัวแปร State จะมาที่นี่ภายหลัง

  @override
  Widget build(BuildContext context) {
    // เมธอด build อยู่ที่นี่แล้ว!
    return Scaffold(
```

```
      appBar: AppBar(
        title: const Text('ปฏิบัติการบทที่ 7 - State'),
        backgroundColor: Colors.purple,
      ),
      body: const Center(
         child: Text('เริ่มทดลอง State ที่นี่!'),
      ),
    );
  }
}
```

6. คำแนะนำ: รันแอป
    - ผลลัพธ์ที่คาดหวัง (UI): แอปควรมีลักษณะเหมือนเดิมทุกประการ เราเพียงแค่เปลี่ยนโครงสร้างภายในเพื่อรองรับ state แต่ยังไม่ได้เพิ่ม state ที่เปลี่ยนแปลงใดๆ

แบบฝึกหัดที่ 2: สร้างตัวนับด้วย setState

1. คำแนะนำ: เพิ่มตัวแปร state ภายในคลาส _MyHomePageState (แต่อยู่นอกเมธอด build) ประกาศตัวแปร integer _counter และกำหนดค่าเริ่มต้นเป็น 0

```
class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0; // ตัวแปร state ของเรา

  @override
  Widget build(BuildContext context) { ... }
}
```

2. คำแนะนำ: แสดงตัวแปร state แก้ไขวิดเจ็ต Text ภายใน Center ของ body เพื่อแสดงค่าของ _counter อย่าลืมลบ const ออกจากวิดเจ็ต Text และบรรพบุรุษของมัน (Center, Padding หากคุณเพิ่ม) เนื่องจากค่าตอนนี้เป็นไดนามิก ใช้ string interpolation

```
// ภายใน Center ของ body ของเมธอด build
body: Center( // ไม่สามารถเป็น const ได้อีกต่อไป
   child: Text( // ไม่สามารถเป็น const ได้
     'ค่าตัวนับ: $_counter', // แสดงตัวแปร state
     style: const TextStyle(fontSize: 24), // Style สามารถเป็น const ได้
   ),
),
```

3. คำแนะนำ: เพิ่มเมธอดเพิ่มค่า ภายใน _MyHomePageState สร้างเมธอด _incrementCounter ที่เพิ่มค่า _counter *ภายใน* การเรียก setState

```dart
class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() { // เรียก setState เพื่อกระตุ้นการ rebuild
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) { ... }
}
```

4. คำแนะนำ: เพิ่มปุ่มเพื่อกระตุ้นเมธอด เพิ่ม FloatingActionButton ให้กับ Scaffold ตั้งค่า property onPressed ให้เรียกเมธอด _incrementCounter ของคุณ กำหนด Icons.add เป็น child

```dart
// ภายใน Scaffold ของเมธอด build
return Scaffold(
  appBar: AppBar(...),
  body: Center(...), // ตามที่แก้ไขด้านบน
  floatingActionButton: FloatingActionButton(
    onPressed: _incrementCounter, // เรียกเมธอดเมื่อกด
    tooltip: 'Increment',
    child: const Icon(Icons.add),
  ),
);
```

5. คำแนะนำ: รันแอป แตะ floating action button หลายครั้ง
   - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม): แอปเริ่มต้นแสดง "ค่าตัวนับ: 0" ทุกครั้งที่คุณแตะปุ่ม '+' ตัวเลขที่แสดงบนหน้าจอจะเพิ่มขึ้น (1, 2, 3,...) สิ่งนี้ยืนยันว่า setState กำลังกระตุ้นการ rebuild UI อย่างถูกต้องด้วยค่า _counter ที่อัปเดตแล้ว

     Image Placeholder แสดง UI แอปตัวนับ สถานะเริ่มต้น 0

     (หลังจากการแตะ)

     Image Placeholder แสดง UI แอปตัวนับ สถานะ 3

แบบฝึกหัดที่ 3: วิดเจ็ตโต้ตอบได้ (เปลี่ยนสี)

1. คำแนะนำ: สร้าง StatefulWidget ใหม่ สร้างไฟล์ใหม่ lib/color_changer.dart ภายในนั้น กำหนด StatefulWidget ชื่อ ColorChanger
2. คำแนะนำ: เพิ่ม state สำหรับสี ในคลาส _ColorChangerState เพิ่มตัวแปร state _currentColor ประเภท Color กำหนดค่าเริ่มต้นเป็น Colors.blue
3. คำแนะนำ: สร้างเมธอดเปลี่ยนสี เพิ่มเมธอด _changeColor ที่ใช้ setState เพื่อสุ่มเลือกสีใหม่ (เช่น จากลิสต์ที่กำหนดไว้ล่วงหน้า เช่น [Colors.red, Colors.green, Colors.yellow, Colors.purple]) และกำหนดให้กับ _currentColor

```dart
import 'dart:math'; // จำเป็นสำหรับ Random
import 'package:flutter/material.dart';

class ColorChanger extends StatefulWidget {
  const ColorChanger({super.key});
  @override
  State<ColorChanger> createState() => _ColorChangerState();
}

class _ColorChangerState extends State<ColorChanger> {
  Color _currentColor = Colors.blue; // สถานะเริ่มต้น
  final List<Color> _availableColors = [
    Colors.red, Colors.green, Colors.yellow, Colors.purple, Colors.orange,
Colors.teal
  ];
  final Random _random = Random();

  void _changeColor() {
    setState(() {
      _currentColor = _availableColors[_random.nextInt(_availableColors.length)];
    });
  }

  @override
  Widget build(BuildContext context) {
    // UI จะมาที่นี่
    return Container(); // Placeholder
  }
}
```

4. คำแนะนำ: สร้าง UI ในเมธอด build ของ _ColorChangerState ส่งคืนวิดเจ็ต GestureDetector GestureDetector สามารถตรวจจับการแตะได้ ตั้งค่า property onTap ให้เรียก _changeColor ตั้งค่า child เป็น Container ที่มีขนาดคงที่ (เช่น กว้าง/สูง 150) และตั้งค่า property color ของ Container เป็นตัวแปร state _currentColor เพิ่ม Text ที่จัดกึ่งกลางภายใน container ว่า "แตะฉัน!"

```dart
// ภายในเมธอด build ของ _ColorChangerState
@override
Widget build(BuildContext context) {
  return GestureDetector( // ตรวจจับการแตะ
    onTap: _changeColor, // เรียกเมธอดเมื่อแตะ
    child: Container(
      width: 150,
```

```
      height: 150,
      color: _currentColor, // ใช้ตัวแปร state สำหรับสี
      alignment: Alignment.center, // จัดข้อความกึ่งกลาง
      child: const Text(
        'แตะฉัน!',
        style: TextStyle(color: Colors.white, fontWeight: FontWeight.bold),
      ),
    ),
  );
}
```

5. คำแนะนำ: ใช้วิดเจ็ตใหม่ กลับไปที่ lib/main.dart Import color_changer.dart แทนที่เนื้อหา body ของ Scaffold ใน MyHomePage ด้วยวิดเจ็ต ColorChanger ใหม่ของคุณ (ห่อด้วย Center หากต้องการ) ลบตัวแปร state และเมธอดตัวนับออกจาก _MyHomePageState หากยังอยู่

```
// ใน lib/main.dart
import 'package:flutter/material.dart';
import 'color_changer.dart'; // Import วิดเจ็ตใหม่

// ... (คลาส MyApp) ...

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key});
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('ปฏิบัติการบทที่ 7 - State'),
        backgroundColor: Colors.purple,
      ),
      body: const Center( // จัดกึ่งกลาง ColorChanger
        child: ColorChanger(), // ใช้วิดเจ็ตใหม่
      ),
      // ลบ FloatingActionButton หากยังอยู่
    );
  }
}
```

6. คำแนะนำ: รันแอป แตะสี่เหลี่ยมสีซ้ำๆ
    - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม): คุณเห็นสี่เหลี่ยมสี (เริ่มต้นเป็นสีน้ำเงิน) พร้อมข้อความ "แตะฉัน!" ทุกครั้งที่คุณแตะสี่เหลี่ยม สีพื้นหลังของมันจะเปลี่ยนแบบสุ่มเป็นสีใดสีหนึ่งที่กำหนดไว้ในลิสต์
      Image Placeholder แสดงสี่เหลี่ยมสีน้ำเงินเริ่มต้นพร้อมข้อความ "แตะฉัน!"
      (หลังจากการแตะ)
      Image Placeholder แสดงสี่เหลี่ยมสีเขียวพร้อมข้อความ "แตะฉัน!"

แบบฝึกหัดที่ 4: การใช้ initState

1. คำแนะนำ: แก้ไขวิดเจ็ต ColorChanger (lib/color_changer.dart) เราต้องการตั้งค่าสี *เริ่มต้น* แบบสุ่มแทนที่จะเริ่มด้วยสีน้ำเงินเสมอ
2. คำแนะนำ: Override เมธอด initState() ใน _ColorChangerState อย่าลืมเรียก super.initState()
3. คำแนะนำ: *ภายใน* initState() *หลังจาก* เรียก super.initState() เพิ่มตรรกะเพื่อสุ่มเลือกสีจาก _availableColors และกำหนดให้กับ _currentColor ลบการกำหนดค่าเริ่มต้น (= Colors.blue) ออกจากการประกาศตัวแปร

```dart
// ภายใน _ColorChangerState
class _ColorChangerState extends State<ColorChanger> {
  late Color _currentColor; // ใช้ 'late' จะถูกกำหนดค่าใน initState
  final List<Color> _availableColors = [ /* ... สี ... */ ];
  final Random _random = Random();

  @override
  void initState() {
    super.initState(); // เรียก super ก่อน
    // ตั้งค่าสีเริ่มต้นแบบสุ่ม
    _currentColor = _availableColors[_random.nextInt(_availableColors.length)];
    print('สีเริ่มต้นที่ตั้งใน initState: $_currentColor'); // สำหรับดีบัก
  }

  void _changeColor() { /* ... เหมือนเดิม ... */ }

  @override
  Widget build(BuildContext context) { /* ... เหมือนเดิม ... */ }
}
```

4. คำแนะนำ: รันแอป (คุณอาจต้อง Hot Restart หรือรีสตาร์ทเต็มรูปแบบ) รันหลายๆ ครั้ง
    - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม): แต่ละครั้งที่แอปเริ่มทำงาน สี่เหลี่ยมควรปรากฏด้วยสีเริ่มต้นแบบสุ่ม *ที่แตกต่างกัน* (เลือกจากลิสต์ของคุณ) แทนที่จะเริ่มด้วยสีน้ำเงินเสมอ การแตะยังคงเปลี่ยนสีแบบสุ่ม

แบบฝึกหัดที่ 5: การใช้ dispose (ฝึกแนวคิด)

1. คำแนะนำ: แก้ไขวิดเจ็ต ColorChanger อีกครั้ง เราจะเพิ่ม TextEditingController เพื่อฝึกการ dispose มัน แม้ว่าเราจะไม่ได้ใช้มันสำหรับ input ในแล็บนี้ก็ตาม

2. คำแนะนำ: Import dart:async หากคุณต้องการเพิ่ม Timer ในภายหลัง แม้ว่าจะไม่จำเป็นอย่างเคร่งครัดสำหรับแค่ controller
3. คำแนะนำ: ภายใน _ColorChangerState ประกาศ late final TextEditingController _controller; late final บ่งชี้ว่ามันจะถูกกำหนดค่าเริ่มต้นครั้งเดียว (ใน initState) และจะไม่ถูกกำหนดค่าใหม่
4. คำแนะนำ: ใน initState() กำหนดค่าเริ่มต้นให้ controller: _controller = TextEditingController();
5. คำแนะนำ: Override เมธอด dispose() ภายใน dispose() เรียก _controller.dispose() เพื่อล้างข้อมูล จากนั้นเรียก super.dispose()

```
// ภายใน _ColorChangerState
class _ColorChangerState extends State<ColorChanger> {
  late Color _currentColor;
  final List<Color> _availableColors = [ /* ... */ ];
  final Random _random = Random();

  late final TextEditingController _controller; // ประกาศ controller

  @override
  void initState() {
    super.initState();
    _currentColor = _availableColors[_random.nextInt(_availableColors.length)];
    _controller = TextEditingController(); // กำหนดค่าเริ่มต้น controller
    print('initState: Controller ถูกสร้างแล้ว');
  }

  @override
  void dispose() {
    print('dispose: กำลัง dispose controller');
    _controller.dispose(); // Dispose controller
    super.dispose(); // เรียก super dispose เป็นลำดับสุดท้าย
  }

  void _changeColor() { /* ... */ }

  @override
  Widget build(BuildContext context) { /* ... */ }
}
```

6. คำแนะนำ: รันแอป นำทางออกจากหน้าจอหากเป็นไปได้ (แม้ว่าในแอปง่ายๆ นี้ การปิดแอปอาจเป็นวิธีเดียวที่จะกระตุ้น dispose) ตรวจสอบ debug console
    - ผลลัพธ์ที่คาดหวัง (พฤติกรรม): แอปทำงานเหมือนเดิม เมื่อ _ColorChangerState ถูกลบออกอย่างถาวร (ซึ่งอาจเกิดขึ้นเฉพาะเมื่อแอปปิดสนิทในการตั้งค่าง่ายๆ นี้ หรือหากคุณสร้างการนำทางเพื่อลบมันออก) คุณควรเห็นข้อความ "dispose: กำลัง dispose controller" พิมพ์ออกมาใน console ซึ่งบ่งชี้ว่าเมธอดล้างข้อมูลถูกเรียกแล้ว นี่แสดงให้เห็นรูปแบบที่ถูกต้องสำหรับการล้างทรัพยากร

ยินดีด้วย! คุณได้สร้าง stateful widgets, ใช้ setState เพื่ออัปเดต UI ตามการเปลี่ยนแปลง state, สร้างเมธอด lifecycle พื้นฐาน (initState, dispose), และแยกความแตกต่างระหว่าง StatelessWidget และ StatefulWidget ได้สำเร็จแล้ว